CS251: Cryptocurrencies and Blockchain Technologies **Fall 2025**

Programming Project #4

Due: 11:59pm on Tue., **Nov. 18, 2025** Submit via Gradescope code: **KDJ7NE**

In this project you will build an on-chain Ethereum Wallet. Recall that Ethereum supports two types of accounts:

- EOAs (Externally Owned Accounts): controlled by a private key, that could be held for instance in a hardware wallet,
- contracts: controlled by code.

When people talk about wallets, they typically refer to software or hardware that holds private keys and can control any number of EOAs. These EOAs can hold funds, tokens, deploy contracts, and generally start transactions. But it is also possible to implement a wallet as a smart contract. The smart contract wallet sends and receives assets based on hard-coded access control rules. The benefit of an on-chain wallet is that these rules can be much more flexible than the access control rules governing an EOA address.

In this project you will build a smart contract wallet, which is initially trivial, but we will progressively add more features to it to make the notion of *programmable money* more tangible.

This project consists of three required checkpoints, and one extra credit checkpoint. Each checkpoint has some required functionality for you to implement. It also comes with Solidity unit tests that check if your contract meets the requirements.

1 Getting started: installing Foundry

For CLI/Solidity only development, we will be using Foundry. Follow instructions at getfoundry.sh:

```
# first, download the foundry installer, foundryup
curl -L https://foundry.paradigm.xyz | bash

# then run it to install forge, cast, anvil, etc
foundryup
```

Foundry's job is to let us:

- build our Solidity source code (that includes getting and invoking the right `solc` version, installing libraries, etc.)
- run unit and fuzz tests, also written in Solidity
- measure the performance of our code (in gas)
- deploy contracts using scripts

Optional (not needed for this project). Scaffold-eth2 can provide you with a web app that lets you interact with the contract visually or with JavaScript:

- install the requirements at https://docs.scaffoldeth.io/quick-start/installation
- run `npx create-eth@latest`
- pick `foundry` as the Solidity framework

The rest of this document will assume a Foundry-only minimal setup, with unit tests written in Solidity and no need for Node or any of the JavaScript ecosystem.

2 Checkpoint 0: basic ownership and handling native currency

Implement a smart contract in `src/0-basic/BasicWallet.sol` such that:

- it has an `owner() public view return(address)` method that returns the current owner of the wallet
- the default value for `owner` is the address who deployed the contract
- it can receive ether from other addresses
- it can send ether to other addresses

Your code should pass the tests in `test/0-basic/BasicWallet.t.sol`. We suggest studying the testing code carefully to learn what is expected of your code. You can run these tests with:

```
# these will fail initially, they run against your BasicWallet.sol implementation
forge test --match-contract Checkpoint0
```

Follow up questions (please submit your answers):

- what happens when call `transferEth` with an amount greater than the contract's balance?
- what happens when the `recipient` of a transfer does not exist?
- what happens when the `recipient` of a transfer is a contract?
- why do we use `msg.sender` and not `tx.origin` to perform the authorization check?
- does `owner` have to be an EOA? if not, how?
- what happens if we lose access to the key that controls `owner`?
- why do we transfer ether out of the contract with a low level call and not Solidity's `transfer`?
- what happens if we send ERC-20 tokens (e.g. USDC) to our wallet address?

References:

- Sending and Receiving Ether (Solidity docs)
- https://docs.soliditylang.org/en/v0.8.30/security-considerations.html#tx-origin
- https://diligence.consensys.io/blog/2019/09/stop-using-soliditys-transfer-now/
- https://gist.github.com/0xkarmacoma/4f206a46dedc6da6808c1ccdef3262d0

3 Checkpoint 1: handling ERC-20 tokens

Implement a smart contract in `src/1-erc20/ERC20Wallet.sol` such that:

- it can receive ERC-20 tokens (hint: not much to do here)
- it can transfer tokens
- it can manage token approvals

Your code should pass the tests in `test/1-erc20/ERC20Wallet.t.sol`. If you look at the deceptively simple ERC-20 token standard, you may think there is not much to it, but the tests showcase some quirks you may encounter with real tokens in the wild. We suggest studying the testing code carefully to learn what is expected of your code. You can run the tests with:

```
# these will fail initially, they run against your ERC20Wallet.sol implementation
forge test --match-contract Checkpoint1
```

Follow-up questions (please submit your answers):

- what would happen if we didn't handle token quirks properly?
- how would we know about ERC-20 tokens owned by the wallet? (e.g. to display in a UI)
- what about `decimals()`? does the wallet need to be concerned with these?

References:

- https://eips.ethereum.org/EIPS/eip-20
- https://ethereum.org/developers/docs/standards/tokens/erc-20/
- https://github.com/d-xo/weird-erc20
- https://andrej.hashnode.dev/integrating-arbitrary-erc-20-tokens
- https://docs.openzeppelin.com/contracts/5.x/api/token/erc20#SafeERC20
- USDT token contract

4 Checkpoint 2: basic multisig

A limitation of our current setup is that a wallet owned by a single EOA has all the same limitations as the EOA itself:

- if the private key for the EOA is lost, then control over the smart contract is lost (including all its assets)
- if the private key for the EOA is compromised, the hacker instantly gets access to all the assets in the smart contract (in addition to the assets in the EOA itself)

But the real power of a smart contract wallet is that it can have more complex authorization logic. For instance, instead of relying on a single EOA, it could use different setups:

- 1-of-2: 2 admin addresses (e.g. a hot wallet and a hardware wallet), any address can approve transactions. Works against lost keys, but not compromised keys.
- 2-of-2: 2 admin addresses required (e.g. desktop wallet + mobile wallet for confirmation) for every transaction. Works against compromised keys, but increases the risk to lose a key.
- 2-of-3, 3-of-5, etc: addresses both risks, but adds complexity

For this checkpoint, we will focus on hardcoding a simple 2-of-3 policy with a fixed set of admins. You can store authorizations on chain.

Implement the following changes to `src/2-multisig/MultisigWallet.sol`:

- pass three admin addresses in the constructor instead of relying on the deployer
- implement an `approve(bytes)` function that admins can invoke to authorize an action encoded in `bytes` and emits an `Approved(address admin, bytes action)` event
- implement an `execute(bytes)` function that anyone can invoke (!) to execute the action encoded in `bytes` as long as it is approved by 2 or more admins, and also counts as an approval when invoked by an admin

Because `execute(bytes)` counts as an approval when invoked by an admin, the following two flows should be supported:

```
approve(bytes) // by admin1
approve(bytes) // by admin2
execute(bytes) // by anyone
```

or:

```
approve(bytes) // by admin1
execute(bytes) // by admin2
```

Your code should pass the tests in `test/2-multisig/MultisigWallet.t.sol`. We suggest studying the testing code carefully to learn what is expected of your code. You can run the tests with:

```
forge test --match-contract Checkpoint2
```

Follow-up questions (please submit your answers):

- how would you implement flexible policies (updatable owners, updatable threshold)? think about failure scenarios like `threshold == 0` or `threshold > admins.length`
- what is the point of emitting events in `approve` and `execute`?

5 Checkpoint 3: integration with Aave (optional: extra credit)

Our smart contract wallet is functional, but one drawback is that the assets it holds are just sitting there, not being productive. It would be nice if they could automatically be deposited into a protocol that would earn yield.

As of 2025, Aave is the leading lending/borrowing protocol on Ethereum. In this checkpoint, we will see how it works to deposit ERC20 tokens and ETH into Aave. Note that until now the

tests were entirely self-contained, but since Aave is an external protocol, we will use mainnet fork testing. This means that in order to run the tests locally, you need to have an Ethereum mainnet RPC url (e.g. via Infura or Alchemy) and then invoke the tests with:

```
MAINNET_RPC_URL=<your-rpc-url> forge test --mc Checkpoint3
```

The contract will also need use the real mainnet addresses of various tokens and Aave endpoints, see `src/3-aave/AaveAddresses.sol`.

Implement the integration in `src/3-aave/AaveSupplier.sol` with the following low-level interface:

```
    // deposit ERC20 tokens owned by this contract to Aave
    function depositERC20(address asset, uint256 amount) external;

    // withdraw ERC20 tokens from Aave, send them to the recipient
    function withdrawERC20(address asset, uint256 amount, address recipient)
    external;
```

and the following higher-level interface:

```
    // wraps the contract balance of ETH to WETH and deposits it to Aave
    function depositEth() external payable;

    // withdraws the contract balance of WETH from Aave, unwraps it to ETH
    // and sends it to the recipient
    function withdrawEth(address recipient) external returns (uint256);
```

The tests in `test/3-aave/AaveSupplier.t.sol` focus on the Aave integration this time rather than auth or the multisig functionality. You can run the tests with:

```
forge test --match-contract Checkpoint3
```

6 Submitting your code

We will be using Gradescope for submission. Please submit a flat zip file with your files

- `BasicWallet.sol`
- `ERC20Wallet.sol`
- `MultisigWallet.sol`
- `AaveSupplier.sol` (optional, extra credit)

and a single PDF containing your answers to the follow-up questions (please start each checkpoint on a new page). Your submission will be graded on whether it passes all the provided tests, as well as the submitted writeup.

Acknowledgment. We thank Daniel "karmacoma" Reynaud for his hard work in developing this project.